
Vom Sparproblem zur Schleife

Das Konzept einer Schleife brauchen wir, sobald in einem Programm dieselbe Berechnung mehrfach hintereinander ausgeführt werden muss. Wir schauen uns das an einem konkreten Sparproblem an.

Eine Person legt 1000 Euro auf ein Sparkonto mit 5 % Zinsen pro Jahr. Sie möchte wissen, in wie vielen Jahren sich ihr Guthaben verdoppelt hat – also 2000 Euro erreicht sind.

Aufgabe 2. Schätzen Sie zuerst alleine, ohne zu rechnen: Wie viele Jahre dauert es ungefähr, bis sich das Guthaben verdoppelt hat?

Meine Schätzung: _____ Jahre

Tauschen Sie sich anschließend mit Ihrer Sitznachbarin oder Ihrem Sitznachbarn aus: Wer hat höher geschätzt, wer niedriger? Im Anschluss sammeln wir die Schätzungen im Plenum.

Aufgabe 3. Wir prüfen die Schätzungen nach. Rechnen Sie zuerst alleine die ersten Jahre aus und tragen Sie die Werte in die Tabelle ein. Vergleichen Sie Ihre Tabelle dann mit Ihrer Sitznachbarin oder Ihrem Sitznachbarn. Im Anschluss vervollständigen wir die Tabelle gemeinsam im Plenum.

Jahr	Guthaben Jahresanfang	Zinsen	Guthaben Jahresende
1	1000,00 Euro		
2			
3			
...			

Aufgabe 4. Schauen Sie auf die Tabelle. Welcher Rechenschritt wiederholt sich in jeder Zeile? Wann hören Sie auf zu rechnen? Formulieren Sie zuerst alleine in eigenen Worten, dann tauschen Sie sich mit Ihrer Sitznachbarin oder Ihrem Sitznachbarn aus. Im Anschluss einigen wir uns im Plenum auf eine gemeinsame Formulierung.

Solange _____

wiederhole _____

Aufbau einer while-Schleife

Eine *Iteration* ist die geplante Wiederholung eines Anweisungsblocks. Statt jede Anweisung nur einmal auszuführen, prüft das Programm vor jedem Durchlauf eine Bedingung: Ist sie *wahr*, wird der Block erneut ausgeführt; ist sie *falsch*, springt das Programm hinter die Schleife und macht dort weiter.

In Java leiten wir diese Form der Wiederholung mit dem Schlüsselwort **while** (englisch für „solange“) ein. Direkt dahinter folgt in runden Klammern die Bedingung – derselbe Typ von Ausdruck, den Sie aus der Selektion bereits kennen. Solange die Bedingung wahr ist, wird der Anweisungsblock in den geschweiften Klammern immer wieder ausgeführt.

```
1 while (Bedingung) {  
2     // Anweisungen, die wiederholt werden sollen  
3     // Achtung: hier muss sich etwas ändern,  
4     //         das die Bedingung beeinflusst!  
5 }
```

- **while:** _____
- *runde* Klammern: _____
- *geschweifte* Klammern: _____
- innerhalb des Schleifenkörpers: _____

Wichtig: Wenn sich innerhalb des Schleifenkörpers nichts verändert, was die Bedingung beeinflusst, läuft die Schleife endlos weiter.

Bausteine einer Schleife

Eine **while**-Schleife besteht immer aus denselben vier Bausteinen. Bevor Sie eine Schleife schreiben, hilft es, sich diese Bausteine bewusst zu machen – in dieser Reihenfolge.

Baustein	Funktion	Beispiel
Startwert	Wo beginnen wir?	<code>int jahre = 0;</code>
Bedingung	Wann hören wir auf?	<code>guthaben < sparziel</code>
Schleifenkörper	Was wird wiederholt?	Zinsen aufrechnen
Veränderung	Was bringt uns dem Abbruch näher?	<code>jahre = jahre + 1;</code>

Mein Sparplan-Programm

Das Programm liest das Ausgangskapital, den Zinssatz und das Sparziel ein. Mit einer `while`-Schleife rechnet es Jahr für Jahr die Zinsen auf das Guthaben auf und zählt dabei die Jahre mit. Sobald das Sparziel erreicht ist, endet die Schleife und das Programm gibt die Anzahl der benötigten Jahre sowie das aktuelle Guthaben aus.

```

1  import java.util.Scanner;
2
3  public class Sparplan {
4      public static void main(String[] args) {
5          Scanner s = new Scanner(System.in);
6
7          System.out.print("Ausgangskapital in Euro: ");
8          double guthaben = s.nextDouble();
9
10         System.out.print("Zinssatz in Prozent: ");
11         double zinssatz = s.nextDouble();
12
13         System.out.print("Sparziel in Euro: ");
14         double sparziel = s.nextDouble();
15
16         int jahre = 0;
17
18         /*
19          * Hier folgt Ihre while-Schleife.
20          * Übersetzen Sie die zuvor erarbeitete
21          * "Solange ..., wiederhole ..." -Formulierung in Java.
22          */
23
24         System.out.println("Nach " + jahre + " Jahren ist das Sparziel erreicht.");
25         System.out.println("Aktuelles Guthaben: " + guthaben + " Euro");
26
27         s.close();
28     }
29 }

```

Aufgabe 5. Schreiben Sie den gemeinsam entwickelten Programmteil hier sauber ab. Achten Sie besonders auf die Einrückung der Anweisungen innerhalb der geschweiften Klammern. Sie macht den Code lesbar und hilft Ihnen selbst, den Überblick zu behalten.

Laden Sie als Grundlage die in den Aufgaben erwähnten Programme vom Schulserver.

Aufgabe 1 (*Bakterienkultur, **).

Eine Bakterienkultur verdoppelt sich jede Stunde. Erweitern Sie das Programm `Bakterien.java`, das die Startanzahl der Bakterien einliest und ausgibt, nach wie vielen Stunden die Anzahl die Million überschritten hat.

Aufgabe 2 (*Schulden abzahlen, **).

Eine Person hat Schulden. Jeden Monat zahlt sie 10 % ihrer aktuellen Restschuld zurück. Erweitern Sie das Programm `Schulden.java`, das die anfängliche Schuldenhöhe einliest und ausgibt, nach wie vielen Monaten die Restschuld unter 100 Euro gesunken ist.

Hinweis. Verwenden Sie für das Guthaben den Datentyp `double`, weil bei der prozentualen Verringerung Nachkommastellen entstehen.

Aufgabe 3 (*Sparplan mit Sparrate, ***).

Erweitern Sie das Programm `Sparplan.java` aus der Stunde um eine jährliche Sparrate. Lassen Sie den Benutzer zusätzlich eingeben, wie viel Euro am Anfang jedes Jahres eingezahlt werden. Diese Einzahlung wird zusammen mit dem bisherigen Guthaben verzinst.

Das Programm gibt am Ende aus, nach wie vielen Jahren das Sparziel erreicht ist und wie hoch das Guthaben dann ist.

Aufgabe 4 (*Bankenvergleich, ****).

Erweitern Sie das Programm `Bankenvergleich.java`. Es liest Ihr Ausgangskapital und Ihr Sparziel ein. Anschließend berechnet es für zwei Banken die Anzahl der Jahre bis zum Sparziel:

- Bank A bietet 2 % Zinsen pro Jahr.
- Bank B bietet 3,5 % Zinsen pro Jahr.

Geben Sie für beide Banken die Anzahl der Jahre aus. Zusätzlich soll das Programm angeben, um wie viele Jahre Bank B schneller ist als Bank A.

Tipp. Sie können *zwei* `while`-Schleifen verwenden – eine pro Bank. Achten Sie darauf, dass jede Schleife mit ihrem eigenen Startguthaben beginnt. Alternativ könnten Sie auch die Bedingungen in einer Schleife verknüpfen.

Aufgabe 5 (*Zahlen, ****).

Sie haben gestern eine Datei `Zahlen.java` modifiziert. Schreiben Sie ein äquivalentes Programm, das nur eine `while`- und keine `for`-Schleife verwendet.